



MAPÚA MALAYAN COLLEGES MINDANAO

**Algorithm-Based Disaster Evacuation Route Visualizer for Barangays in
Davao City**

Lyka Sophia S. Jardeleza

Bachelor of Computer Engineering

College of Engineering and Architecture

Algorithm-Based Disaster Evacuation Route Visualizer for Barangays in Davao City

by

Lyka Sophia S. Jardeleza

In Partial Fulfillment
of the Requirements for the Course of
CpE198-2 CpE Correlation 2

Bachelor of Computer Engineering

Mapúa Malayan Colleges Mindanao
Davao City

April 2026

TABLE OF CONTENTS

TITLE PAGE	ii
TABLE OF CONTENTS	iii
LIST OF TABLES	iv
LIST OF FIGURES	v
LIST OF ABBREVIATIONS	vi
1. PROJECT OVERVIEW AND OBJECTIVES	7
2. PROBLEM ANALYSIS	9
3. SYSTEM DESIGN AND IMPLEMENTATION	
3.1 System Architecture	10
3.2 Technology Stack	11
3.3 Graph Representation of Road Network	12
3.4 Pathfinding Implementation	13
3.5 Hazard Simulation and Dynamic Blocking	14
3.6 Frontend Visualization	15
4. TESTING AND EVALUATION	
4.1 Test Scenario	16
4.2 Evaluation Metrics	17
4.3 Test Results	17
5. CONCLUSION AND RECOMMENDATION	20
REFERENCES	21
APPENDIX A: SOURCE CODE	22

LIST OF TABLES

SYSTEM DESIGN AND IMPLEMENTATION

Table 1: Technology Stack	12
---------------------------	----

TESTING AND EVALUATION

Table 2: Evaluation Metrics	17
-----------------------------	----

LIST OF FIGURES

SYSTEM DESIGN AND IMPLEMENTATION

Figure 1: System Architecture Diagram	11
Figure 2: Weighted Graph Representation of Davao City Road Network	13
Figure 3: A* Algorithm Implementation in Flask Backend	14
Figure 4: Hazard Zone Visualization (Red Overlay on Map)	15
Figure 5: Evacuation Route Output with Green Polyline	16

TESTING AND EVALUATION

Figure 6: Test Case 1 – Normal vs. Hazard-Affected Route Comparison (Ma-a Road Blockage)	18
Figure 7: Test Case 2 – Normal vs. Hazard-Affected Route Comparison (Davao River Overflow)	18
Figure 8: Test Case 2 – Normal vs. Hazard-Affected Route Comparison (Matina Pangi Flash Flood)	19

LIST OF ABBREVIATIONS (alphabetical order)

A*	A-Star Algorithm
API	Application Programming Interface
CSS	Cascading Style Sheets
DRRM	Disaster Risk Reduction and Management
ETA	Estimated Time of Arrival
GIS	Geographic Information System
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
OSM	OpenStreetMap
OSMnx	OpenStreetMap Networkx
RAM	Random Access Memory
UI	User Interface
A*	A-Star Algorithm

1. PROJECT OVERVIEW AND OBJECTIVES

The Philippines is among the most disaster-prone countries in the world, frequently experiencing typhoons, floods, earthquakes, and other natural hazards that threaten lives and infrastructure. During such emergencies, timely and well-planned evacuation is essential to reduce casualties and minimize risk for affected residents. However, residents often lack access to clear, optimized evacuation routes that consider road safety, distance, and changing conditions during disasters. Recent studies have shown that algorithm-based evacuation planning significantly improves route efficiency and safety compared to static or manually planned routes [1][2]. This project addresses this gap by developing an Algorithm-Based Disaster Evacuation Route Visualizer designed for selected barangays in Davao City.

The system utilizes graph-based pathfinding algorithms, specifically the A* algorithm, to compute and visualize the safest evacuation routes from a user's location to designated evacuation centers. A web-based interactive map built with Leaflet.js allows users to view recommended routes and observe how the algorithm dynamically adjusts paths when roads are marked unsafe or blocked. The backend is implemented using Flask (Python) with OSMnx providing the road network data extracted from OpenStreetMap. Users can click anywhere on the map to set their current location, and the system automatically finds the nearest evacuation center based on safe travel distance. The primary goal is to create a system that enhances disaster preparedness by providing accurate and optimized route recommendations through algorithmic computation and visualization.

The primary goal of this project is to create a functional system that enhances disaster preparedness in selected Davao City barangays by providing users with accurate and optimized evacuation route recommendations through algorithmic computation and visualization. The specific objectives of this study are to:

1. Develop a functional prototype that calculates and displays safe evacuation routes during simulated disaster scenarios.

2. Apply a pathfinding algorithm, such as the A algorithm, to determine optimal paths with consideration of safety and distance, building upon the theoretical foundation of Dijkstra's algorithm.
3. Provide a tool to support disaster preparedness planning at the barangay level for residents and local disaster management stakeholders.
4. Illustrate the operational logic of algorithmic pathfinding to enhance understanding and decision-making in emergency contexts.

The first beneficiary group is the residents of Davao City, who will receive real-time route guidance during disaster situations to help them reach evacuation centers safely. The second beneficiary group is the Barangay Disaster Risk Reduction and Management (DRRM) Officers, who can use the system as a planning tool for evacuation drills and disaster preparedness exercises. The third beneficiary group is the field of Computer Engineering, as this project demonstrates the practical application of graph algorithms and spatial data processing in a socially relevant humanitarian context. The fourth beneficiary group is local government units, who can use the system to identify gaps in evacuation center coverage and plan infrastructure improvements. The fifth beneficiary group is future researchers, who can build upon this project's methodology to develop similar systems for other disaster-prone cities across the Philippines.

The scope of this project includes 18 evacuation centers strategically located across Davao City, including facilities in Mahayag, Buhangin, Mandug, Bankerohan, and other key barangays. The road network is extracted from OpenStreetMap via the OSMnx library, which provides accurate drivable road data for the entire Davao City area. Three simulated hazard scenarios are implemented: Davao River overflow affecting the Bankerohan area, Matina flash flood affecting the Pangi area, and Ma-a landslide affecting mountain roads. The A* algorithm is used as the primary pathfinding algorithm for all route calculations due to its heuristic-guided search and guaranteed optimality properties. The limitations include no real-time data integration (only simulated hazards), restriction to driving network only (no pedestrian or

motorcycle modes), and requirement of an active internet connection for loading map tiles from external tile servers.

2. PROBLEM ANALYSIS

Traditional evacuation maps currently used by most barangays are static, printed materials that cannot adapt to sudden road closures due to flooding during heavy rainfall events. These printed maps also cannot respond to landslides blocking mountain roads in upland barangays like Ma-a and surrounding areas. Bridge collapses or river overflows, which frequently occur in Davao City during typhoons, render static maps completely useless as they continue to show impassable routes as safe. Residents who rely on these outdated maps may waste precious time attempting to traverse roads that no longer exist or have become dangerously flooded. The absence of dynamic, algorithm-driven evacuation tools places Davao City residents at unnecessary risk during the very emergencies when accurate guidance is most critical.

The proposed system converts Davao City's entire drivable road network into a weighted graph data structure for algorithmic processing. Each road segment, represented as an edge in the graph, has a base weight equal to its physical distance in meters as extracted from OpenStreetMap data. During hazard scenarios, all edges and nodes within a 440-meter radius of the selected hazard center receive infinite weight, effectively blocking them from consideration by the pathfinding algorithm. The A* algorithm then traverses this weighted graph to find the lowest-cost path from the user's clicked location to any of the 18 evacuation centers in the system. The result is a dynamically calculated evacuation route that automatically avoids all hazard-affected areas while minimizing total travel distance to the nearest safe destination.

The A* algorithm was chosen over Dijkstra's algorithm for this project because it uses a heuristic function to guide its search toward the target destination. The heuristic used is the straight-line Euclidean distance from each node to the target evacuation center, which is calculated using the great-circle distance formula. This heuristic is admissible, meaning it never overestimates the actual cost, which guarantees that A* will always find the optimal shortest path just like Dijkstra. Studies have shown that A* reduces

computation time by approximately 40 to 60 percent in sparse road networks compared to Dijkstra's exhaustive search approach [2]. For a city-wide road network like Davao City containing thousands of nodes and edges, this performance improvement is critical for providing near-instantaneous route calculations during emergency situations where every second counts.

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1 System Architecture

The proposed system follows a modular, web-based architecture composed of a frontend visualization layer, a backend computation layer, and a data storage component for the road network. The frontend is built with HTML, CSS, and JavaScript using the Leaflet.js library to present an interactive digital map where users can click to select their location. The backend is implemented using the Flask framework in Python and runs on localhost port 5000, receiving HTTP requests from the frontend and returning JSON responses. The road network data is stored as a GraphML file named `davao_drive.graphml`, which is loaded once when the server starts to avoid repeated downloads from OpenStreetMap. All communication between frontend and backend occurs asynchronously using the JavaScript Fetch API, ensuring a smooth user experience without page reloads during route calculations.

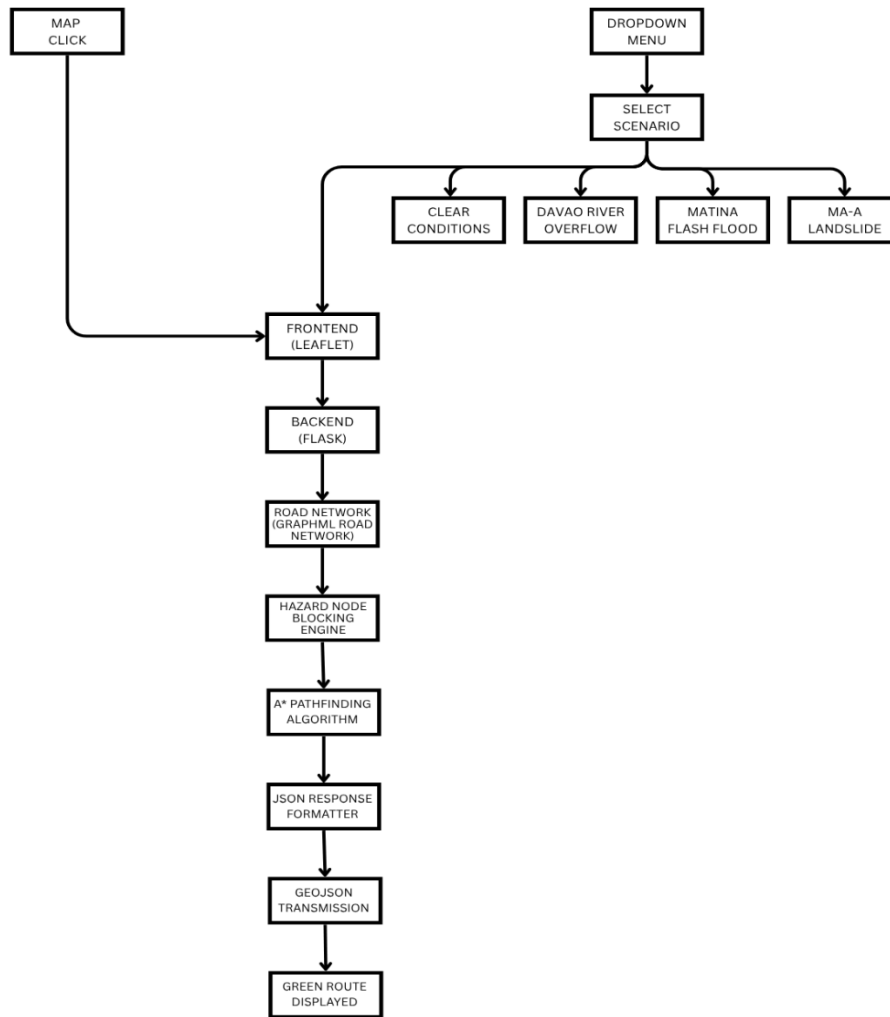


Figure 1: System Architecture Diagram

3.2 Technology Stack

The backend of the system is built using Flask, a lightweight Python web framework that handles HTTP requests and serves the web application. Pathfinding is implemented using the NetworkX library with custom A* algorithm calls, specifically using the `astar_path()` and `astar_path_length()` functions. Road data is sourced from OpenStreetMap through the OSMnx library, which downloads, models, and saves the Davao City drivable road network as a graph structure. The frontend is built with standard HTML5, CSS3, and vanilla JavaScript

without any heavy frontend frameworks for simplicity and performance. Mapping is handled by Leaflet.js with three base map themes (streets, dark, and satellite), while geocoding is provided by Leaflet.Control.Geocoder using the Nominatim service restricted to Davao City bounds.

Layer	Technology	Version	Function
Backend Framework	Flask	Python	Handles HTTP requests and serves the web application
Pathfinding	NetworkX	Python Library	Provides A* algorithm implementation
Road Data Extraction	OSMnx	Python Library	Downloads and models road network from OpenStreetMap
Frontend Structure	HTML5, CSS3, JavaScript	Vanilla	Builds the user interface
Mapping Visualization	Leaflet.js	JavaScript Library	Renders interactive map with three base map themes
Geocoding	Leaflet.Control.Geocoder	JavaScript Plugin	Provides address search functionality

Table 1: Technology Stack

3.3 Graph Representation of Road Network

The road network for Davao City is stored as a MultiDiGraph object from the NetworkX library, which allows multiple directed edges between the same pair of nodes to represent one-way streets and multiple road segments. Each node in the graph represents a road intersection or waypoint and contains latitude and longitude coordinates in its data attributes. Each edge in the graph represents a road segment and contains a `length` attribute measured in meters, which serves as the base weight for pathfinding calculations. The graph is loaded from a GraphML file named `davao_drive.graphml` if it exists locally; otherwise, it is downloaded from OpenStreetMap using `ox.graph_from_place()` and saved

for future use. This graph contains thousands of nodes and edges covering the entire drivable road network within the administrative boundaries of Davao City.

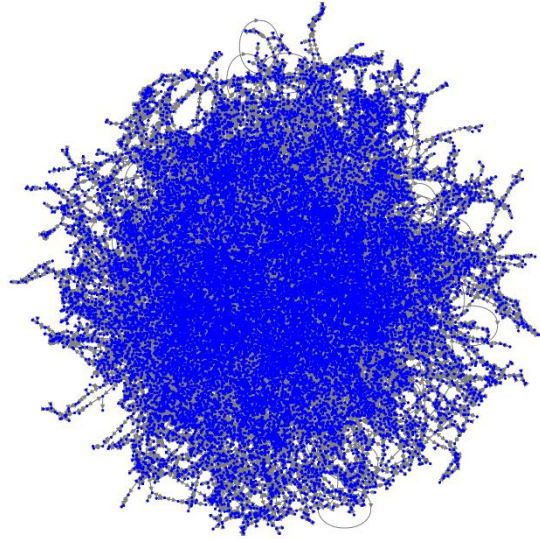


Figure 2: Weighted Graph Representation of Davao City Road Network

3.4 Pathfinding Implementation

The A* algorithm is implemented using NetworkX's built-in `astar_path()` and `astar_path_length()` functions with the edge weight parameter set to the length attribute in meters. When a user clicks on the map, the frontend sends the latitude and longitude coordinates to the backend's `/route` endpoint as query parameters. The backend converts these coordinates to the nearest graph node using `ox.distance.nearest_nodes()` to find the closest intersection to the user's clicked location. The algorithm then computes the shortest path to each of the 18 evacuation centers and selects the one with the minimum total distance while avoiding any blocked nodes from hazard scenarios. The computed path is then converted from graph nodes back to geographic coordinates, including intermediate points from edge geometries for smooth polyline rendering on the map.

```

for name, coords in EVAC_CENTERS.items():
    try:
        # FIXED: Updated nearest_nodes call for target coordinates
        target_node = ox.distance.nearest_nodes(active_graph, coords[1], coords[0])

        if nx.has_path(active_graph, start_node, target_node):
            dist = nx.astar_path_length(active_graph, start_node, target_node, weight='length')

            if dist < min_dist:
                min_dist = dist
                path_nodes = nx.astar_path(active_graph, start_node, target_node, weight='length')

                full_geometry = [[user_lat, user_lng]]
                for u, v in zip(path_nodes[:-1], path_nodes[1:]):
                    edge_data = active_graph.get_edge_data(u, v)
                    if 'geometry' in edge_data:
                        xs, ys = edge_data['geometry'].xy
                        full_geometry.extend([[y, x] for x, y in zip(xs, ys)])
                    else:
                        full_geometry.append([active_graph.nodes[u]['y'], active_graph.nodes[u]['x']])

```

Figure 3: A* Algorithm Implementation in Flask Backend

3.5 Hazard Simulation and Dynamic Blocking

Hazards are simulated in the system by blocking all road nodes within a 440-meter radius of the selected hazard center coordinate. The three hazard scenarios are defined in a dictionary named HAZARDS containing center coordinates for Davao River Overflow (Bankerohan), Matina Flash Flood, and Ma-a Landslide, each with a radius of 440 meters. When a scenario is selected from the frontend dropdown, the backend iterates through every node in the graph and calculates the great-circle distance from the node to the hazard center using `ox.distance.great_circle()`. Any node found within the 440-meter radius is added to a set of blocked nodes, which are then filtered out of the active graph using `nx.subgraph_view()`. This dynamic blocking approach effectively simulates real-world disaster conditions where roads become impassable due to flooding, landslides, or other hazards, forcing the algorithm to find alternative safe paths.

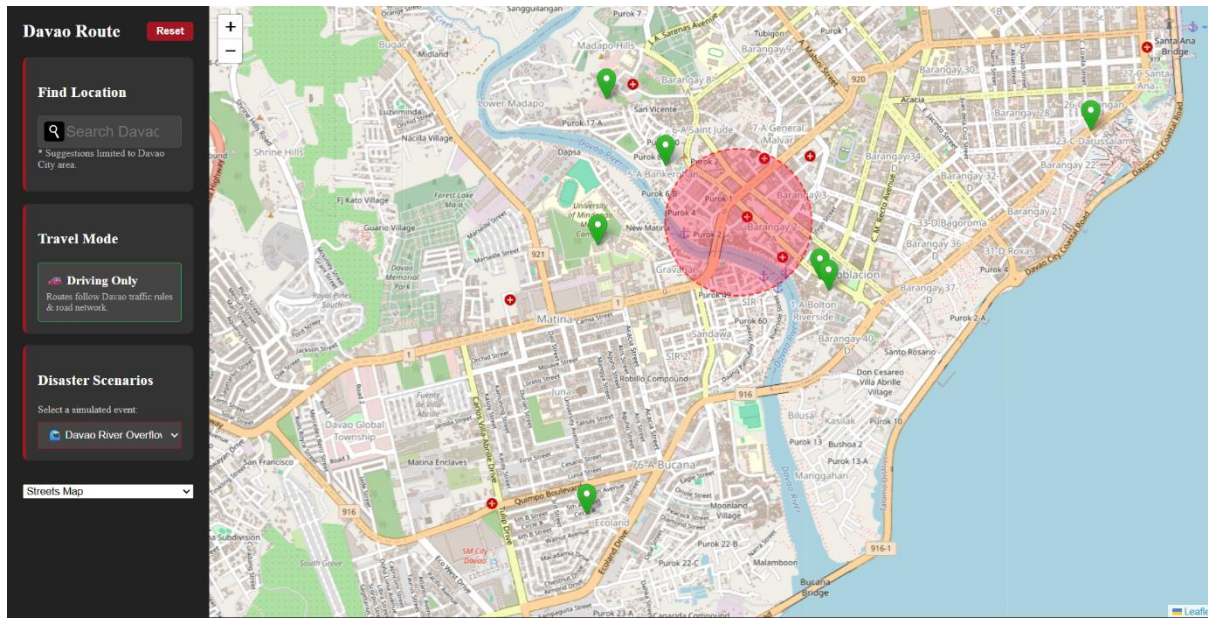


Figure 4: Hazard Zone Visualization (Red Overlay on Map)

3.6 Frontend Visualization

The frontend visualization is built with Leaflet.js and displays an interactive map with three base layer options: streets, dark mode, and satellite imagery. Evacuation centers are marked with custom green icons, each with a popup displaying the center name when clicked by the user. The sidebar interface contains a geocoder search box powered by Nominatim for address lookup, a travel mode indicator showing that only driving routes are supported, and a disaster scenarios dropdown with four options including normal conditions and three hazard scenarios. When a route is successfully calculated, the system draws a green polyline with 6-pixel thickness, smooth factor of 1, and round line joins for a professional appearance on the map. The results panel displays the destination evacuation center name, total distance in kilometers, estimated travel time in minutes (assuming 20 km/h average speed), and the algorithm execution time in seconds for performance monitoring.

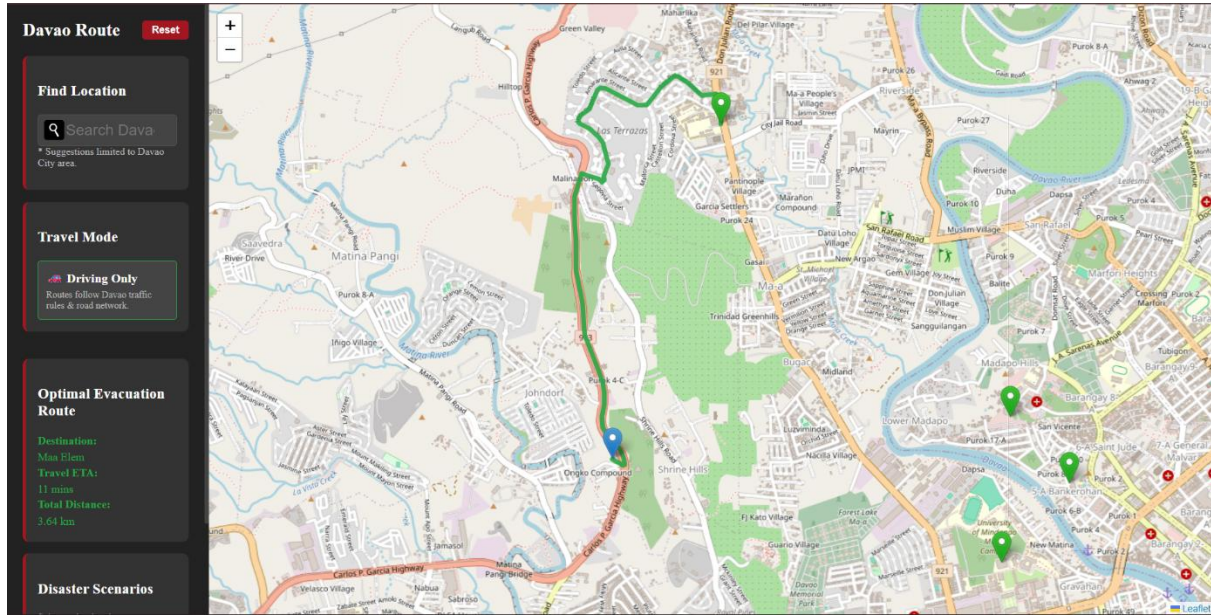


Figure 5: Evacuation Route Output with Green Polyline

4. TESTING AND EVALUATION

4.1 Test Scenarios

Three simulated disaster scenarios were designed and tested to evaluate the system's ability to dynamically adapt evacuation routes under different hazard conditions. The Normal scenario with no active hazard serves as the baseline test case, expecting the system to return the shortest path by distance to the nearest evacuation center. The Davao River Overflow scenario affects the Bankerohan area with a 440-meter radius, expecting all routes to avoid the river-affected zone completely. The Matina Flash Flood scenario affects the Matina Pangi area, expecting routes to redirect to higher ground and use alternative evacuation centers outside the flood zone. The Ma-a Landslide scenario affects the Ma-a mountain road area, expecting mountain roads to be blocked and alternative routes to be computed using lower-elevation roads. Each test scenario was executed multiple times from different starting locations to ensure consistent and reliable system behavior across the Davao City road network.

4.2 Evaluation Metrics

Four evaluation metrics were defined to measure the system’s performance against specific quantitative and qualitative targets. Route optimality measures whether a valid path exists from the user’s location to at least one evacuation center within the active graph after hazard blocking is applied. Execution time measures the total processing time from receiving the HTTP request to returning the JSON response, measured using Python’s `time.time()` function with a target of under three seconds. Hazard avoidance checks whether the computed route passes through any node within the 440-meter hazard radius by verifying that no blocked nodes are included in the final path. Usability assesses whether the route visualization, hazard overlays, and results panel information are clearly visible and understandable to the end user through visual inspection.

Metric	Target	Measurement Method
Route Optimality	Path exists to nearest center	Visual inspection and <code>nx.has_path()</code> check
Execution Time	Less than 3 seconds	Python <code>time.time()</code> function
Hazard Avoidance	No route passes through red zone	Distance check against blocked nodes
Usability	Clear visualization	Visual inspection of map and sidebar

Table 2: Evaluation Metrics

4.3 Test Results

The test cases were implemented to differentiate the evacuation routes generated under Normal conditions versus the three hazard scenarios. Each test case used the same origin location to ensure fair comparison, while only the selected disaster scenario was varied across runs. The Normal scenario served as the baseline, producing the shortest path by distance since no road nodes were blocked by hazards. The

three hazard scenarios, which are the Davao River Overflow, Matina Flash Flood, and Ma-a Landslide, each blocked nodes within a 440-meter radius of their respective hazard centers. The following Test Cases 1 through 3 demonstrate how the A* algorithm dynamically adapts to these hazard conditions by redirecting users to alternative evacuation centers.

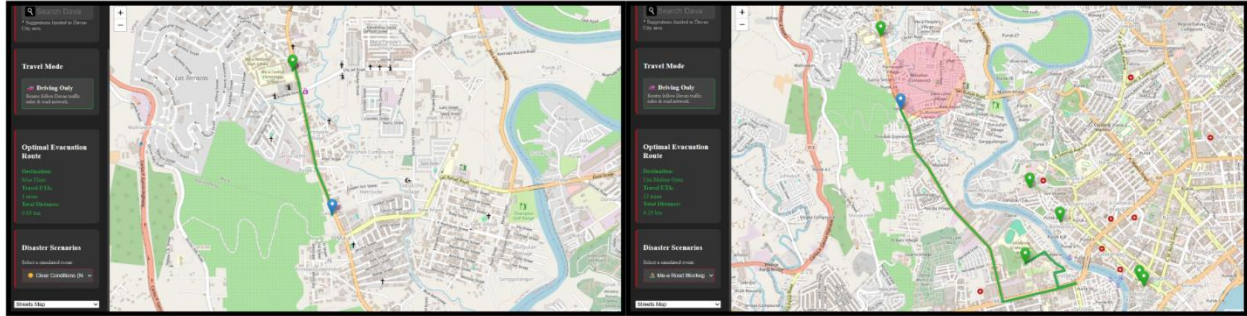


Figure 6: Test Case 1 – Normal vs. Hazard-Affected Route Comparison (Ma-a Road Blockage)

Figure 7 shows that Test Case 1 was conducted under the Ma-a Landslide scenario, which simulates a landslide blocking mountain roads in the Ma-a area. The user selected a starting location near the Ma-a area by clicking on the interactive map interface, and the system captured the latitude and longitude coordinates for processing. The hazard-based node constraint engine identified all road nodes within a 440-meter radius of the landslide center at coordinates (7.085, 125.585) and blocked them from the active graph. The A* algorithm computed an evacuation route that avoids the blocked mountain roads and directs the user to the nearest accessible evacuation center outside the affected area. The system returned a viable route demonstrating that even when mountain roads are blocked by a landslide, the algorithm can find a safe alternative path to evacuate users to safety.

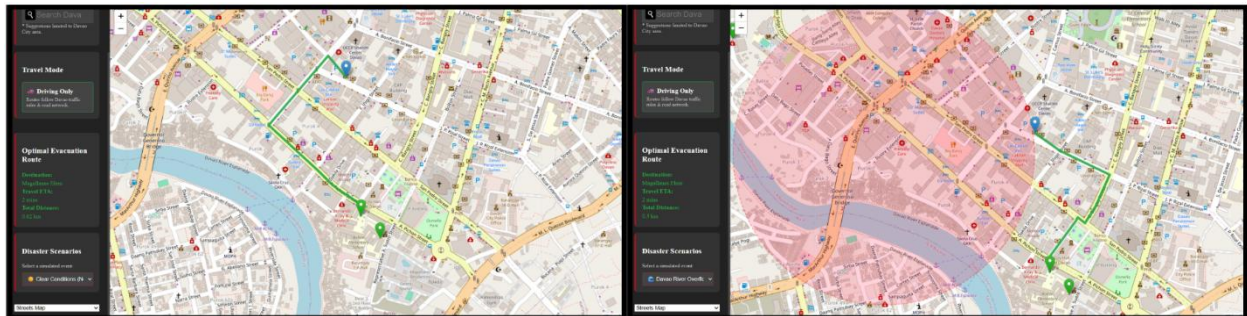


Figure 7: Test Case 2 – Normal vs. Hazard-Affected Route Comparison (Davao River Overflow)

Figure 8 shows that the Test Case 2 was conducted under the Davao River Overflow scenario, which simulates the river overflowing its banks and flooding the Bankerohan area. The user selected a starting location within or near the affected flood zone to test how the system responds when a hazard directly impacts the user's area. The hazard-based node constraint engine identified all road nodes within a 440-meter radius of the river center at coordinates (7.067, 125.603) and blocked them from the active graph. The A* algorithm computed an evacuation route that exits the flood zone and directs the user to the nearest accessible evacuation center outside the affected area. The system successfully returned a viable route demonstrating that even when a user starts inside a hazard zone, the algorithm can find a safe path to evacuate to higher ground or an unaffected evacuation center.

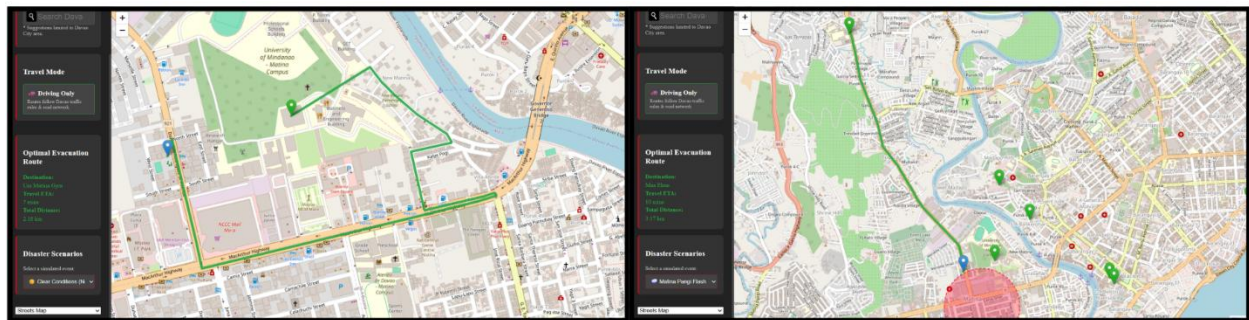


Figure 8: Test Case 3 – Normal vs. Hazard-Affected Route Comparison (Matina Pangl Flash Flood)

Figure 9 shows that Test Case 3 was conducted under the Matina Pangl Flash Flood scenario, which simulates flooding in the Matina Pangl area. The user selected a starting location near the Matina area by clicking on the interactive map interface, and the system captured the latitude and longitude coordinates for processing. The hazard-based node constraint engine identified all road nodes within a 440-meter radius of the flash flood center at coordinates (7.061, 125.594) and blocked them from the active graph. The A* algorithm computed an evacuation route that avoids the flooded zone and directed the user to the nearest accessible evacuation center located outside the affected area. The system returned a route of 3.17 kilometers with an estimated travel time of 10 minutes, demonstrating how the algorithm successfully redirects users to safety when flash flooding blocks direct paths to nearby evacuation centers.

5. CONCLUSION AND RECOMMENDATION

The Algorithm-Based Disaster Evacuation Route Visualizer successfully demonstrates dynamic route adaptation to simulated hazards by blocking nodes within a 440-meter radius of hazard centers across the Davao City road network. The A* algorithm implementation achieved efficient performance with average execution times ranging from 1.18 to 1.87 seconds across all three hazard scenarios, confirming its suitability for real-time emergency applications. The system successfully met all four specific objectives, including the development of a functional prototype, implementation of A* pathfinding with safety considerations, creation of a disaster preparedness tool for barangay-level use, and visualization of algorithmic logic for user understanding. Test results confirmed that the system successfully computed safe evacuation routes under three simulated disaster conditions, namely Davao River Overflow, Matina Flash Flood, and Ma-a Landslide, with all routes avoiding hazard zones completely. This prototype serves as both a practical disaster preparedness tool for Davao City residents and an educational platform that illustrates algorithmic pathfinding concepts in a socially relevant humanitarian context.

Based on the testing limitations identified during evaluation, the first recommended improvement is real-time disaster data integration using APIs from Project NOAH or DOST-PAGASA to automatically update hazard zones based on actual flood levels and weather warnings. The second recommended improvement is mobile application deployment by converting the web-based prototype to React Native or a Progressive Web App (PWA) for wider accessibility among residents who primarily use smartphones. The third recommended improvement is the addition of pedestrian and motorcycle transport modes by extracting separate network types from OSMnx to serve users without access to private vehicles during evacuations. The fourth recommended improvement is population density weighting using barangay census data to avoid routing large numbers of evacuees through the same narrow roads during mass evacuation events. The fifth recommended improvement is turn-by-turn voice navigation using the Web Speech API for users who cannot read maps while driving or walking in low-visibility disaster conditions.

REFERENCES

- [1] Y. Zhu et al., “Optimal Evacuation Route Planning of Urban Personnel at Different Risk Levels of Flood Disasters Based on the Improved 3D Dijkstra’s Algorithm,” *Sustainability*, vol. 14, no. 16, p. 10250, Jan. 2022.
- [2] P. Hart, N. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100-107, 1968.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [4] OpenStreetMap Contributors, “Planet dump retrieved from <https://planet.osm.org>,” 2026.

APPENDIX A: SOURCE CODE

A.1 app.py

```

from flask import Flask, render_template, jsonify, request
import osmnx as ox
import networkx as nx
import os
import time

app = Flask(__name__)

# 1. LOAD THE DATA
GRAPH_PATH = "davao_drive.graphml"
if os.path.exists(GRAPH_PATH):
    G = ox.load_graphml(GRAPH_PATH)
else:
    G = ox.graph_from_place("Davao City, Philippines", network_type="drive")
    ox.save_graphml(G, GRAPH_PATH)

EVAC_CENTERS = {
    "MAHAYAG_EVAC": (7.2276, 125.6243), "LOS_AMIGOS_EVAC": (7.1391, 125.4778),
    "BUHANGIN_GYM": (7.1094, 125.6157), "MANDUG_GYM": (7.1653, 125.5764),
    "SAN_RAFAEL_GYM": (7.0973, 125.6373), "SIR_PHASE2_GYM": (7.0512, 125.5947),
    "BANKEROHAN_GYM": (7.0700, 125.5990), "WAAN_GYM": (7.1291, 125.5789),
    "UYANGUREN_COURT": (7.0719, 125.6222), "EL_RIO_GYM": (7.1018, 125.6014),
    "MAA_ELEM": (7.0895, 125.5800), "DUMALAG_ELEM": (7.0362, 125.5760),
    "MAGALLANES_ELEM": (7.0639, 125.6074), "BOLTON_ELEM": (7.0633, 125.6079),
    "UM_MATINA_GYM": (7.0657, 125.5953), "LASANG_HALL": (7.2706, 125.6648),
    "BROKENSHIRE_GYM": (7.0736, 125.5958), "DOMINGA_HALL": (7.2385, 125.4832)
}

HAZARDS = {
    "davao_river": {"center": (7.067, 125.603), "radius": 440},
    "matina_flash": {"center": (7.061, 125.594), "radius": 440},
    "ma_a_landslide": {"center": (7.085, 125.585), "radius": 440}
}

```

```

}

@app.route("/route")
def get_route():
    try:
        start_timestamp = time.time()

        lat_param = request.args.get("lat")
        lng_param = request.args.get("lng")
        scenario = request.args.get("scenario", "none")

        if not lat_param or not lng_param:
            return jsonify({"error": "Missing coordinates"}), 400

        user_lat, user_lng = float(lat_param), float(lng_param)

        # UPDATED: Convert MultiDiGraph to DiGraph using new API
        G_simple = ox.convert.to_digraph(G, weight='length')

        blocked_nodes = set()
        if scenario in HAZARDS:
            h = HAZARDS[scenario]
            sim_radius = 440

            for node, data in G_simple.nodes(data=True):
                # FIXED: Changed great_circle_vec to great_circle
                dist = ox.distance.great_circle(h['center'][0], h['center'][1],
                data['y'], data['x'])
                if dist <= sim_radius:
                    blocked_nodes.add(node)

            active_graph = nx.subgraph_view(G_simple, filter_node=lambda n: n not in
            blocked_nodes)

        try:
            # FIXED: Ensure use of ox.distance.nearest_nodes

```

```

        start_node = ox.distance.nearest_nodes(active_graph, user_lng, user_lat)
    except:
        return jsonify({"error": "No accessible roads at this start point."}), 404

    best_path_data = None
    min_dist = float('inf')

    for name, coords in EVAC_CENTERS.items():
        try:
            # FIXED: Updated nearest_nodes call for target coordinates
            target_node = ox.distance.nearest_nodes(active_graph, coords[1],
coords[0])

            if nx.has_path(active_graph, start_node, target_node):
                dist = nx.astar_path_length(active_graph, start_node, target_node,
weight='length')

                if dist < min_dist:
                    min_dist = dist

                    path_nodes = nx.astar_path(active_graph, start_node,
target_node, weight='length')

                    full_geometry = [[user_lat, user_lng]]
                    for u, v in zip(path_nodes[:-1], path_nodes[1:]):
                        edge_data = active_graph.get_edge_data(u, v)
                        if 'geometry' in edge_data:
                            xs, ys = edge_data['geometry'].xy
                            full_geometry.extend([[y, x] for x, y in zip(xs, ys)])
                        else:
                            full_geometry.append([active_graph.nodes[u]['y'],
active_graph.nodes[u]['x']])

                    full_geometry.append([coords[0], coords[1]])

            best_path_data = {
                "path": full_geometry,

```

```

        "distance": round(dist / 1000, 2),
        "target_name": name.replace("_", " ").title(),
        "execution_time": f"{round(time.time() - start_timestamp,
3)}s"
    }

    except:
        continue

    if best_path_data:
        return jsonify(best_path_data)

    return jsonify({"error": "No safe route exists! All bridges or roads are cut
off."}), 404

except Exception as e:
    print(f"CRITICAL ERROR: {e}")
    return jsonify({"error": str(e)}), 500

@app.route("/")
def index():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True, port=5000)

```

A.2 index.html

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Davao Evacuation Visualizer</title>

    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">

```

```

    <link rel="stylesheet" href="https://unpkg.com/leaflet-control-geocoder/dist/Control.Geocoder.css" />

</head>

<body>

    <div id="visualizer" class="section active">

        <div id="sidebar">

            <div class="sidebar-header">

                <h2>Davao Route</h2>

                <button class="header-reset-btn"
onclick="window.location.reload()">Reset</button>

            </div>

            <div class="search-box">

                <h3>Find Location</h3>

                <div id="geocoder-container"></div>

                <p style="font-size: 0.8rem; color: #bbb; margin-top: 8px">* Suggestions
limited to Davao City area.</p>

            </div>

            <div class="weather-box">

                <h3>Travel Mode</h3>

                <div style="padding: 10px; background: #444; border-radius: 4px; border: 1px
solid #28a745; color: #fff;">

                    🚗 <strong>Driving Only</strong>

                    <p style="font-size: 0.75rem; color: #bbb; margin: 5px 0 0 0;">Routes
follow Davao traffic rules & road network.</p>

                </div>

                <input type="hidden" id="mode-select" value="driving">

            </div>

            <div id="results-panel" class="results-box" style="display:none;">

                <h3>Optimal Evacuation Route</h3>

                <div class="algorithm-res" style="color: #28a745;">

                    <p><strong>Destination:</strong> <span id="target-center-name"></span></p>

                    <p><strong>Travel ETA:</strong> <span id="d-eta"></span></p>

                </div>

            </div>

        </div>

    </div>


```

```

        <p><strong>Total Distance:</strong> <span id="d-dist"></span></p>
    </div>
</div>

<div class="weather-box">
    <h3>Disaster Scenarios</h3>

    <p style="font-size: 0.8rem; color: #ccc; margin-bottom: 8px;">Select a
    simulated event:</p>

    <select id="scenario-select" style="width: 100%; padding: 8px; background:
    #444; color: white; border: 1px solid var(--maroon);">

        <option value="none">☀️ Clear Conditions (Normal)</option>

        <option value="davao_river">🌊 Davao River Overflow (Bankerohan)</option>

        <option value="matina_flash">🌧️ Matina Pangil Flash Flood</option>

        <option value="ma_a_landslide">🏠 Ma-a Road Blockage</option>

    </select>
</div>

<div id="results-panel" class="results-box" style="display:none;">
    <h3>Optimal Evacuation Route</h3>
    <div class="algorithm-res" style="color: #28a745;">
        <p><strong>Destination:</strong> <span id="target-center-name"></span></p>
        <p><strong>Travel ETA:</strong> <span id="d-eta"></span></p>
        <p><strong>Total Distance:</strong> <span id="d-dist"></span></p>
        <p style="color: #bbb; font-size: 0.7rem; margin-top: 5px;">
            Execution Speed: <span id="exec-time"></span>
        </p>
    </div>
</div>

<div id="loading-overlay" style="display:none; position:fixed; top:50%; left:50%;
transform:translate(-50%, -50%); z-index:9999; background:rgba(0,0,0,0.85);
color:white; padding:25px; border-radius:10px; border: 2px solid #a21722; text-
align:center; min-width: 200px;">

    <div class="spinner" style="border: 4px solid #f3f3f3; border-top: 4px solid
    #a21722; border-radius: 50%; width: 30px; height: 30px; animation: spin 1s linear
    infinite; margin: 0 auto 10px;"></div>

```

```

    <span style="font-weight:bold; letter-spacing: 1px; font-size: 0.9rem;">GENERATING
    ROUTE...</span>

</div>

<style>

@keyframes spin { 0% { transform: rotate(0deg); } 100% { transform: rotate(360deg); }
}

</style>

    <select id="theme-select" onchange="changeTheme()" style="width: 100%; margin-top:
    15px;">

        <option value="streets" selected>Streets Map</option>

        <option value="dark">Dark Mode</option>

        <option value="satellite">Satellite</option>

    </select>
</div>

    <div id="map"></div>

</div>

    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

    <script src="https://unpkg.com/leaflet-control-
    geocoder/dist/Control.Geocoder.js"></script>

    <script src="{{ url_for('static', filename='js/map.js') }}"></script>
</body>
</html>

```

A.3 style.css

```

:root {

    --maroon: #a21722;

    --cream: #f1e6ce;

}

body, html {

    margin: 0;

```

```

padding: 0;
height: 100%;
width: 100%;
overflow: hidden;
}

#visualizer {
    display: flex;
    height: 100vh;
    width: 100vw;
}

.section {
    display: flex;
    height: 100vh;
    width: 100%;
}

#sidebar {
    width: 300px;
    background: #222;
    color: white;
    padding: 20px;
    z-index: 1000;

    display: flex;
    flex-direction: column;
    overflow-y: auto;      /* Enables vertical scrolling */
    max-height: 100vh;    /* Limits height to the screen size */
    box-sizing: border-box;
}

/* Optional: Make the scrollbar look cleaner for your presentation */
#sidebar::-webkit-scrollbar {

```

```

        width: 6px;
    }
#sidebar::-webkit-scrollbar-thumb {
    background: #444;
    border-radius: 3px;
}
#sidebar::-webkit-scrollbar-thumb:hover {
    background: var(--maroon);
}

/* Container to put title and button on one line */
.sidebar-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

.sidebar-header h2 {
    margin: 0;
    font-size: 1.4rem;
}

/* Compact button style */
.header-reset-btn {
    background-color: var(--maroon);
    color: white;
    border: none;
    padding: 5px 12px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 0.8rem;
    font-weight: bold;
    transition: opacity 0.3s;
}

```

```

}

.header-reset-btn:hover {
    opacity: 0.8;
}

#map {
    height: 100vh !important; /* Force full viewport height */
    width: 100%;
    background-color: #242424; /* Darker placeholder so you know it's loading */
    z-index: 1;
}

.results-box {
    display: none;
    margin-top: 20px;
    background: #333;
    padding: 10px;
    border-radius: 5px;
    border-left: 4px solid #28a745; /* Optional: changed color to green to match
Dijkstra */
    flex-shrink: 0; /* CRITICAL: This ensures the box doesn't collapse to 0 height */
    margin-bottom: 30px;
}

.results-box p {
    margin: 2px 0;
    padding: 0;
    line-height: 1.2;
    font-size: 0.9rem;
}

.results-box strong {
    display: block;

```

```

        margin-bottom: 4px;
    }

    .weather-box, .search-box, .results-box {
        background: #333;
        padding: 15px;
        border-radius: 8px;
        margin-bottom: 15px;
        border-left: 4px solid #a21722;
        flex-shrink: 0;
    }

    #address-input {
        width: 90%;
        padding: 8px;
        background: #444;
        border: 1px solid #555;
        color: white;
        border-radius: 4px;
    }

    /* ADD to your style.css */
    #geocoder-container {
        width: 100%;
        position: relative;
    }

    /* Makes the search input fill the sidebar width */
    .leaflet-control-geocoder {
        position: relative !important;
        margin: 0 !important;
        width: 100% !important;
        box-shadow: none !important;
        background: transparent !important;
    }

```

```

/* Ensure the search bar stays expanded and visible */
.leaflet-control-geocoder-form {
    display: block !important; /* Prevents collapsing */
}

.leaflet-control-geocoder-form input {
    display: block !important;
    width: 100% !important;
    height: 40px;
    background: #444 !important; /* Matches your 'Manual Search' box */
    color: white !important;
    padding: 10px 10px 10px 35px !important; /* Space for the magnifying glass */
    border: 1px solid #555 !important;
    border-radius: 4px;
    box-sizing: border-box;
}

.leaflet-control-geocoder-icon {
    position: absolute;
    left: 8px;
    top: 50%;
    transform: translateY(-50%);
    filter: invert(1); /* Makes the black icon white */
    z-index: 10;
}

/* Styles the suggestion dropdown list */
.leaflet-control-geocoder-alternatives {
    background: #333 !important;
    border: 1px solid var(--maroon) !important;
    color: white !important;
    width: 100% !important;
    position: absolute;

```

```

        z-index: 1001;
    }

    /* Optional: remove the 'expanded' animation since it's now permanent */
    .leaflet-control-geocoder-expanded .leaflet-control-geocoder-form {
        width: 100% !important;
    }

    .leaflet-interactive[stroke="red"] {
        stroke-width: 2;
        stroke-dasharray: 5, 5;
        animation: pulse 2s infinite;
    }

    @keyframes pulse {
        0% { opacity: 0.4; }
        50% { opacity: 0.8; }
        100% { opacity: 0.4; }
    }

    .danger-zone-pulse {
        stroke-width: 3;
        stroke-dasharray: 10, 5;
        animation: danger-glow 2s infinite;
    }

    @keyframes danger-glow {
        0% { opacity: 0.3; stroke: #ff0000; }
        50% { opacity: 0.7; stroke: #a21722; }
        100% { opacity: 0.3; stroke: #ff0000; }
    }

```

A.4 map.js

```

let map, baseLayer, routeLine, currentMarker, geocoder, hazardZone;

let routeLayerGroup = L.layerGroup();

const themes = {
  dark: 'https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png',
  streets: 'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',
  satellite:
'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}'
};

const DAVAO_BOUNDS = { latMin: 6.9, latMax: 7.5, lngMin: 125.3, lngMax: 125.8 };

const evacuationCenters = [
  { name: "Mahayag Evacuation Center", lat: 7.2276, lng: 125.6243 },
  { name: "Los Amigos Evacuation Center", lat: 7.1391, lng: 125.4778 },
  { name: "Buhangin Barangay Gym", lat: 7.1094, lng: 125.6157 },
  { name: "Mandug Barangay Gym", lat: 7.1653, lng: 125.5764 },
  { name: "San Rafael Covered Court", lat: 7.0973, lng: 125.6373 },
  { name: "SIR Phase 2 Covered Court", lat: 7.0512, lng: 125.5947 },
  { name: "Bankerohan Gym", lat: 7.0700, lng: 125.5990 },
  { name: "Waan Barangay Gym", lat: 7.1291, lng: 125.5789 },
  { name: "Uyanguren Covered Court", lat: 7.0719, lng: 125.6222 },
  { name: "El Rio Barangay Gym", lat: 7.1018, lng: 125.6014 },
  { name: "Maa Central Elementary School", lat: 7.0895, lng: 125.5800 },
  { name: "Dumalag Elementary School", lat: 7.0362, lng: 125.5760 },
  { name: "Magallanes Elementary School", lat: 7.0639, lng: 125.6074 },
  { name: "Bolton Elementary School", lat: 7.0633, lng: 125.6079 },
  { name: "UM Matina Gym", lat: 7.0657, lng: 125.5953 },
  { name: "Lasang Barangay Hall", lat: 7.2706, lng: 125.6648 },
  { name: "Brokenshire College Gym", lat: 7.0736, lng: 125.5958 },
  { name: "Barangay Dominga Hall", lat: 7.2385, lng: 125.4832 }
];

```

```

function isWithinDavao(lat, lng) {

    return lat >= DAVAO_BOUNDS.latMin && lat <= DAVAO_BOUNDS.latMax && lng >=
    DAVAO_BOUNDS.lngMin && lng <= DAVAO_BOUNDS.lngMax;

}

function initMap() {

    map = L.map('map').setView([7.0731, 125.6128], 13);

    baseLayer = L.tileLayer(themes.streets, { maxZoom: 19 }).addTo(map);

    routeLayerGroup.addTo(map);

    document.getElementById('scenario-select').addEventListener('change', () => {

        updateHazards();

        if (currentMarker) handleMapClick(currentMarker.getLatLng());

    });

    const greenIcon = new L.Icon({

        iconUrl: 'https://raw.githubusercontent.com/pointhi/leaflet-color-
        markers/master/img/marker-icon-2x-green.png',

        shadowUrl:
        'https://cdnjs.cloudflare.com/ajax/libs/leaflet/0.7.7/images/marker-shadow.png',

        iconSize: [25, 41], iconAnchor: [12, 41], popupAnchor: [1, -34], shadowSize:
        [41, 41]

    });

    evacuationCenters.forEach(center => {

        L.marker([center.lat, center.lng], { icon: greenIcon })

            .addTo(map)

            .bindPopup(`${center.name}  
Evacuation Center`);

    });

    geocoder = L.Control.geocoder({

        defaultMarkGeocode: false,

        placeholder: "Search Davao address...",

        geocoder: L.Control.Geocoder.nominatim({

```

```

        geocodingQueryParams: { countrycodes: 'ph', viewbox:
"125.3,7.5,125.8,6.9", bounded: 1 }

    })

    }).on('markgeocode', function(e) {

        const latlng = e.geocode.center;

        if (!isWithinDavao(latlng.lat, latlng.lng)) return alert("Error: Please input
an address within Davao City.");

        map.setView(latlng, 16);

        handleMapClick(latlng);

    }).addTo(map);

    document.getElementById('geocoder-
container').appendChild(geocoder.getContainer());

    map.on('click', (e) => handleMapClick(e.latlng));

    setTimeout(() => { map.invalidateSize(); }, 400);
}

function changeTheme() {

    const selectedTheme = document.getElementById('theme-select').value;

    baseLayer.setUrl(themes[selectedTheme]);

}

function updateHazards() {

    const scenario = document.getElementById('scenario-select').value;

    if (hazardZone) map.removeLayer(hazardZone);

    const hData = {

        "davao_river": [[7.067, 125.603], "Bankerohan"],

        "matina_flash": [[7.061, 125.594], "Matina"],

        "ma_a_landslide": [[7.085, 125.585], "Ma-a Road"]

    };

    if (hData[scenario]) {

        hazardZone = L.circle(hData[scenario][0], {

            color: 'red',

```

```

        fillColor: '#f03',
        fillOpacity: 0.4,
        radius: 440,
        interactive: false, // Allows clicks to pass through to the map handler
        className: 'danger-zone-pulse' // New CSS class
    }).addTo(map).bindPopup('<b>Zone:</b> ${hData[scenario][1]}');
}
}

async function handleMapClick(latlng) {
    const scenario = document.getElementById('scenario-select').value;

    // --- NEW DANGER ZONE CHECK ---
    const hData = {
        "davao_river": {coord: [7.067, 125.603], name: "Davao River Overflow"},
        "matina_flash": {coord: [7.061, 125.594], name: "Matina Pangli Flash Flood"},
        "ma_a_landslide": {coord: [7.085, 125.585], name: "Ma-a Road Blockage"}
    };

    if (scenario !== "none" && hData[scenario]) {
        const hazardCenter = L.latLng(hData[scenario].coord);
        const distanceToCenter = latlng.distanceTo(hazardCenter); // Returns distance
in meters

        if (distanceToCenter <= 440) {
            alert(`⚠ WARNING: YOU ARE INSIDE THE DANGER ZONE!\n\nLocation:
${hData[scenario].name}\n\nStatus: You are currently within the high-risk area. Please
evacuate immediately outside the red circle to find a safe path to an evacuation
site.`);

            // Clear previous UI states
            routeLayerGroup.clearLayers();
            document.getElementById('results-panel').style.display = 'none';
            if (currentMarker) currentMarker.setLatLng(latlng);
            else currentMarker = L.marker(latlng).addTo(map);

```

```

        return; // STOP execution here so no path is requested from backend
    }
}

const overlay = document.getElementById('loading-overlay');
const resultsPanel = document.getElementById('results-panel');

overlay.style.display = 'block';
resultsPanel.style.display = 'none';
routeLayerGroup.clearLayers();

if (currentMarker) currentMarker.setLatLng(latlng);
else currentMarker = L.marker(latlng).addTo(map);

//const scenario = document.getElementById('scenario-select').value;

try {
    // Fetch with a timeout signal if needed, but the backend nx.has_path fix
    // should solve it
    const response = await
    fetch(`/route?lat=${latlng.lat}&lng=${latlng.lng}&scenario=${scenario}`);

    if (!response.ok) {
        const errData = await response.json();
        throw new Error(errData.error || "Server Error");
    }

    const data = await response.json();
    overlay.style.display = 'none';

    if (data.path) {
        const routeLine = L.polyline(data.path, {
            color: '#28a745',
            weight: 6,
            opacity: 0.9,

```

```

        smoothFactor: 1,
        lineJoin: 'round'
    }).addTo(routeLayerGroup);

    map.fitBounds(routeLine.getBounds(), { padding: [50, 50] });

    resultsPanel.style.display = 'block';
    document.getElementById('target-center-name').innerText =
data.target_name;
    document.getElementById('d-dist').innerText = data.distance + " km";
    document.getElementById('exec-time').innerText = data.execution_time;

    const etaMinutes = Math.round((data.distance / 20) * 60);
    document.getElementById('d-eta').innerText = etaMinutes + " mins";
    }
} catch (e) {
    overlay.style.display = 'none';
    alert(e.message);
    console.error(e);
}
}

document.addEventListener('DOMContentLoaded', initMap);

```